

# Package ‘priceDiscoverY’

September 18, 2026

**Title** Information Share Estimation for Financial Markets with Partially Overlapping Trading Hours

**Version** 0.1.0

**Description** Estimates daily contribution-weighted information shares (CWIS) for futures and spot markets with partially overlapping trading hours. The package estimates Hasbrouck information shares during overlapping sessions, estimates realized variation during single-market sessions, and aggregates period-specific contributions into daily CWIS values. The methodology is based on Dimpfl and Schweikert (2023)  [<doi:10.1016/j.jbankfin.2023.106970>](https://doi.org/10.1016/j.jbankfin.2023.106970).

**License** MIT + file LICENSE

**Encoding** UTF-8

**Roxygen** list(markdown = TRUE)

**Suggests** knitr,  
rmarkdown,  
testthat (>= 3.0.0)

**VignetteBuilder** knitr

**Config/roxygen2/version** 8.1.0

**Imports** highfrequency,  
stats,  
urca,  
xts

**Config/testthat/edition** 3

**Depends** R (>= 3.5)

**LazyData** true

## Contents

|                                             |    |
|---------------------------------------------|----|
| priceDiscoverY-package                      | 2  |
| aggregate_cwis                              | 5  |
| combine_weights                             | 8  |
| common_price                                | 11 |
| cwis                                        | 15 |
| daily_cwis                                  | 21 |
| example_sp500_futures_spot_oneweek_24h_1sec | 25 |
| his                                         | 26 |
| plot.cwis_result                            | 29 |

|                                          |    |
|------------------------------------------|----|
| realized_kernel . . . . .                | 31 |
| realized_variance . . . . .              | 34 |
| split_trading_days . . . . .             | 36 |
| split_trading_sessions . . . . .         | 38 |
| validate_and_standardize_input . . . . . | 40 |
| vecm . . . . .                           | 43 |

|              |           |
|--------------|-----------|
| <b>Index</b> | <b>47</b> |
|--------------|-----------|

---

priceDiscoverRy-package

*priceDiscoverRy: Price discovery in partially overlapping markets*

---

## Description

Provides tools for estimating contribution-weighted information shares (CWIS) for two markets with partially overlapping trading hours. The package combines price discovery during simultaneous trading with price variation generated while only one market is open.

## Background

The same or economically equivalent asset may trade in more than one market. Although the observed prices can temporarily differ because of trading frictions, liquidity, and market-microstructure noise, they are expected to reflect a common underlying efficient price.

Conventional Hasbrouck information shares measure how simultaneously trading markets contribute to innovations in this efficient price. However, they use only periods in which all markets are open. This can omit relevant information when one market trades for longer hours than another.

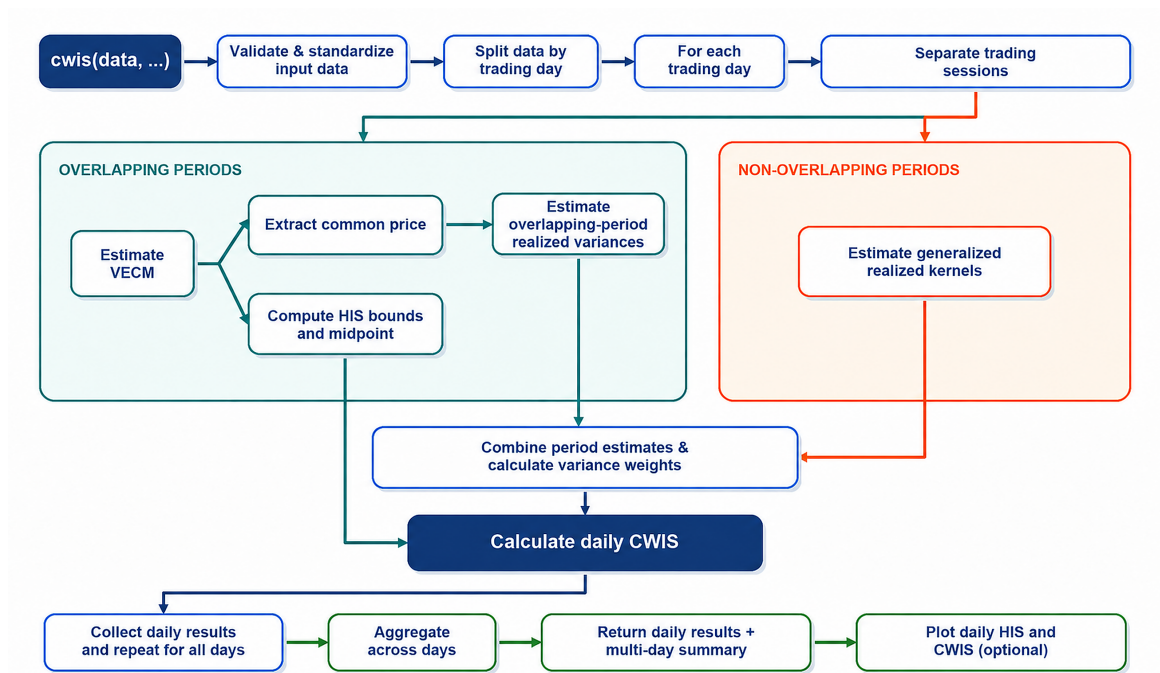
Methods for sequential markets assign price discovery to the market that is open during a non-overlapping period, but they do not directly address a trading day containing both overlapping and non-overlapping sessions.

The contribution-weighted information share proposed by Dimpfl and Schweikert (2023) connects these two cases. It uses Hasbrouck information shares while both markets trade and assigns single-market price discovery to the market that remains open. The different trading periods are weighted by their contributions to total daily efficient-price variation rather than by their duration.

This makes it possible to use price information from the complete trading day and to avoid understating the contribution of a market that continues trading while the other market is closed.

## Method overview

The package follows the computational workflow illustrated below.



The workflow consists of the following steps:

1. Validate and standardize the timestamps and price series.
2. Divide each trading day into overlapping and single-market sessions.
3. Estimate a bivariate vector error-correction model during the overlapping session.
4. Compute the Hasbrouck information-share bounds and their midpoint.
5. Extract the common efficient price and estimate its realized variance.
6. Estimate generalized realized kernels during the single-market sessions.
7. Construct variance-based weights and combine the session-specific estimates.
8. Report daily CWIS estimates and, where requested, aggregate them across trading days.

The main user interface is `cwis()`. Further details on the individual methodological components are provided in the documentation for the corresponding lower-level functions and in the package vignettes.

### Main functions

- `cwis()` calculates CWIS estimates for one or more trading days.
- `daily_cwis()` performs the complete calculation for one trading day.
- `aggregate_cwis()` summarizes estimates and diagnostics across days.
- `vecm()` estimates the overlapping-period VECM.
- `his()` calculates Hasbrouck information shares.
- `common_price()` extracts the common efficient price.
- `realized_kernel()` estimates variation during single-market sessions.
- `combine_weights()` constructs the intraday variance weights.

## Scope and assumptions

The two input price series should represent the same underlying asset, or closely related assets connected by a stable long-run equilibrium relationship. During overlapping trading sessions, the price series are assumed to be non-stationary, cointegrated, and driven by one common efficient-price trend. Prices must be expressed on comparable scales and synchronized before estimation.

The efficient price is assumed to continue evolving when either market is closed. During a single-market session, the open market is assigned the information contribution generated in that session, while the closed market receives no contribution. Session weights are determined by each session's estimated contribution to the daily variation of the efficient price.

Trading days and session boundaries are interpreted in the user-specified time zone. When applying the package to other assets or exchanges, users should verify the time zone, trading-day convention, session definitions, price scaling, sampling frequency, and VECM lag order.

The package estimates a standard Johansen VECM using `urca::ca.jo()`. It implements the general CWIS framework but does not exactly reproduce the high-resolution, HAR-structured VECM used in the empirical analysis of Dimpfl and Schweikert (2023).

## Main references

- Hasbrouck, J. (1995). One security, many markets: Determining the contributions to price discovery. *The Journal of Finance*, 50(4), 1175–1199. doi:10.2307/2329348
- Wang, J. and Yang, M. (2011). Housewives of Tokyo versus the gnomes of Zurich: Measuring price discovery in sequential markets. *Journal of Financial Markets*, 14(1), 82–108. doi:10.1016/j.finmar.2010.08.002
- Hasbrouck, J. (2021). Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 395–430. doi:10.1093/jfinec/nbz027
- Dias, G. F. and Schweikert, K. (2022). Integrated variance estimation for assets traded in multiple venues. *SSRN Working Paper*, 1–48. <https://ssrn.com/abstract=4253762>
- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

## Author(s)

**Maintainer:** Hui Sun <sophia8226@163.com>

Authors:

- Hui Sun <sophia8226@163.com>

Other contributors:

- Thomas Dimpfl [contributor, thesis advisor]
- Karsten Schweikert [contributor, thesis advisor]

## See Also

`cwis()` for the main estimation interface; `daily_cwis()` for the complete single-day calculation; and `aggregate_cwis()` for cross-day summaries.

aggregate\_cwis

*Aggregate daily CWIS estimates***Description**

Aggregates successful daily contribution-weighted information-share estimates and retains diagnostic records for both successful and failed trading days.

The function produces a day-level results table, cross-day summaries of Hasbrouck information shares and CWIS estimates, calculation diagnostics, and summaries of the intraday variance weights.

**Usage**

```
aggregate_cwis(results)
```

**Arguments**

**results** A list containing successful `cwis_day_result` objects produced by `daily_cwis()` and/or failed-day records created by `cwis()`. Each failed record must contain date and error.

**Details**

Dimpfl and Schweikert (2023) calculate the CWIS separately for each trading day and report the time-series mean and standard deviation of the resulting daily estimates. This function implements the corresponding descriptive aggregation.

This function does not recalculate daily information shares, efficient prices, realized variances, or variance weights. It only summarizes values already stored in the supplied daily result objects.

For the cross-day aggregation, let  $x_d$  denote a daily statistic, such as the futures CWIS or the weight of an intraday period. Let  $\mathcal{D}_x$  denote the set of successful days for which  $x_d$  is available, and let  $D_x = |\mathcal{D}_x|$  denote the number of these days. The reported mean is

$$\bar{x} = \frac{1}{D_x} \sum_{d \in \mathcal{D}_x} x_d.$$

When at least two non-missing observations are available, the reported sample standard deviation is

$$s_x = \sqrt{\frac{1}{D_x - 1} \sum_{d \in \mathcal{D}_x} (x_d - \bar{x})^2}.$$

Each available trading day receives the same weight in these calculations. The statistics are not weighted by the number of observations, daily total variance, or session length.

If no non-missing value is available, the mean is `NA_real_`. If fewer than two values are available, the sample standard deviation is `NA_real_`. These outputs are descriptive statistics only. The function does not calculate standard errors, confidence intervals, hypothesis tests, or corrections for serial dependence across trading days.

## Value

A named list with four components:

**daily** A data frame containing one row per supplied trading day, in input order. It contains:

- **date**: trading date;
- **status**: either "ok" or "failed";
- **error**: failure message, or NA for a successful day;
- **his\_futures**, **his\_spot**: daily midpoint HIS estimates;
- **cwis\_futures**, **cwis\_spot**: daily CWIS estimates;
- **overlapping\_weight**: total overlapping-period variance weight;
- **residual\_correlation**: correlation between VECM innovations;
- **zero\_return\_futures**, **zero\_return\_spot**: fractions of zero returns in the overlapping sample;
- **n\_observations**: total daily observations; and
- **n\_overlap**: observations in the overlapping sessions.

Numerical results are NA for failed days.

**information\_share** A data frame with columns **statistic**, **futures**, and **spot**. Its rows contain "HIS mean", "HIS SD", "CWIS mean", and "CWIS SD", calculated across successful days with missing values removed.

**diagnostics** A data frame with columns **statistic** and **value**. It reports the numbers of supplied, successful, and failed days; the mean and sample standard deviation of the overlapping-period weight; and the mean residual correlation.

**variance\_weights** A data frame with columns **period**, **mean**, and **sd**, containing the cross-day mean and sample standard deviation of each period's variance weight. It is empty when no successful day is available.

## Successful and failed days

An element inheriting from class `cwis_day_result` is treated as a successful trading day. Its HIS midpoint, CWIS, variance weights, and diagnostics are included in the cross-day summaries.

Every other element is treated as a failed-day record. A failed day remains in the **daily** table with **status** = "failed" and its error message, but its numerical estimates are recorded as missing and excluded from the summary statistics.

The numbers of supplied, successful, and failed days are reported separately. If no day is successful, the **information\_share** summary contains missing values and the **variance-weight** summary is an empty data frame.

An empty results list is permitted. It produces an empty **daily** table, missing **information\_share** summaries, zero supplied and successful days, and an empty **variance-weight** table.

## Interpretation

**information\_share** compares the conventional midpoint Hasbrouck information share with the full-day CWIS. A difference between their means indicates that non-overlapping trading periods materially change the estimated allocation of price discovery.

As discussed by Putnins (2013), HIS may reflect both the speed at which a market incorporates information and the extent to which its observed price avoids transitory noise. CWIS additionally accounts for efficient-price variation generated while only one market is open.

**variance\_weights** summarizes when efficient-price variation occurs during the trading day. A large mean weight indicates that the corresponding session contributes a large share of average daily price variation; it does not necessarily indicate that the session is long.

## Input requirements

results must be a list. Each successful element must be a complete `cwis_day_result` object produced by `daily_cwis()`. In particular, it must contain:

- `date`;
- `his_midpoint`;
- `cwis`;
- `overlapping_weight`;
- `residual_correlation`;
- `zero_return_fraction`;
- `n_observations`;
- `n_overlap`; and
- `variance_weights`.

A failed-day record must contain a valid date and a character error message.

Successful results should use the same market names and the same named variance-weight components in the same order. Dates should normally be unique. The function preserves the order of results and does not sort the output chronologically.

## References

- Hasbrouck, J. (1995). One security, many markets: Determining the contributions to price discovery. *The Journal of Finance*, 50(4), 1175–1199. doi:10.2307/2329348
- Putniņš, T. J. (2013). What do price discovery metrics really measure? *Journal of Empirical Finance*, 23, 68–83. doi:10.1016/j.jempfin.2013.05.004
- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

## See Also

`daily_cwis()` for calculating the daily HIS, variance weights, and CWIS; `combine_weights()` for constructing daily period weights; `his()` for Hasbrouck information shares; `realized_variance()` and `realized_kernel()` for the underlying session-variance estimates; and `cwis()` for processing multiple trading days and creating failed-day records.

## Examples

```
day_1 <- structure(
  list(
    date = as.Date("2026-01-05"),
    his_midpoint = c(futures = 0.60, spot = 0.40),
    cwis = c(futures = 0.72, spot = 0.28),
    overlapping_weight = 0.70,
    residual_correlation = 0.35,
    zero_return_fraction = c(futures = 0.10, spot = 0.15),
    n_observations = 10000L,
    n_overlap = 7000L,
    variance_weights = c(
      futures_pre = 0.12,
      spot_maintenance = 0.06,
      futures_post = 0.12,
```

```

        overlap_pre = 0.10,
        overlap_core = 0.50,
        overlap_post = 0.10
      )
    ),
    class = "cwis_day_result"
  )

day_2 <- structure(
  list(
    date = as.Date("2026-01-06"),
    his_midpoint = c(futures = 0.55, spot = 0.45),
    cwis = c(futures = 0.68, spot = 0.32),
    overlapping_weight = 0.65,
    residual_correlation = 0.40,
    zero_return_fraction = c(futures = 0.12, spot = 0.18),
    n_observations = 9500L,
    n_overlap = 6500L,
    variance_weights = c(
      futures_pre = 0.15,
      spot_maintenance = 0.05,
      futures_post = 0.15,
      overlap_pre = 0.10,
      overlap_core = 0.45,
      overlap_post = 0.10
    )
  ),
  class = "cwis_day_result"
)

failed_day <- list(
  date = as.Date("2026-01-07"),
  error = "Insufficient observations in one or more sessions."
)

aggregated <- aggregate_cwis(list(day_1, day_2, failed_day))

aggregated$daily
aggregated$information_share
aggregated$diagnostics
aggregated$variance_weights

```

---

combine\_weights

---

Combine period variances into contribution weights

---

## Description

Combines integrated-variance estimates from overlapping and non-overlapping trading sessions and converts them into normalized contribution weights for the daily CWIS calculation.

## Usage

```
combine_weights(overlapping_variance, non_overlapping_variance)
```

## Arguments

overlapping\_variance

A named numeric vector containing integrated- variance estimates for periods in which both markets trade. These values are normally estimated from common-efficient-price returns using `realized_variance()`.

non\_overlapping\_variance

A named numeric vector containing integrated-variance estimates for periods in which only one market trades. These values are normally estimated using `realized_kernel()`.

## Details

Consider a trading day divided into  $H$  mutually exclusive periods  $S_h$ , for  $h = 1, \dots, H$ . Some periods contain simultaneous trading in both markets, whereas others contain trading in only one market.

Following Wang and Yang (2011), information generated during a period is measured by the variation in the efficient price during that period. Dimpfl and Schweikert (2023) extend this variance-share principle to partially overlapping markets.

Let  $\hat{\sigma}_{m,h}^2$  denote the estimated integrated variance of the efficient price  $m$  during period  $S_h$ . The estimated total daily variance is

$$\hat{\sigma}_{m,d}^2 = \sum_{i=1}^H \hat{\sigma}_{m,i}^2.$$

The contribution weight of period  $S_h$  is then

$$w_h = \frac{\hat{\sigma}_{m,h}^2}{\sum_{i=1}^H \hat{\sigma}_{m,i}^2}.$$

Here,  $w_h$  is the proportion of total daily efficient-price variation generated during period  $S_h$ . Provided that all period variances are finite and non-negative and their sum is positive,

$$0 \leq w_h \leq 1 \quad \text{and} \quad \sum_{h=1}^H w_h = 1.$$

For overlapping sessions,  $\hat{\sigma}_{m,h}^2$  is normally calculated from returns of the common efficient price extracted from the VECM. This follows the common-trend approach discussed by Dias and Schweikert (2022) and Dimpfl and Schweikert (2023).

For non-overlapping sessions, only one market price is observed. The corresponding period variance is normally estimated with a realized kernel, following Wang and Yang (2011), Barndorff-Nielsen et al. (2008), and Dimpfl and Schweikert (2023).

The total overlapping weight is the sum of the weights assigned to all subperiods in which both markets are open:

$$w_{\text{overlap}} = w_{\text{overlap\_pre}} + w_{\text{overlap\_core}} + w_{\text{overlap\_post}}.$$

The weights measure contributions to daily efficient-price variation, not proportions of trading time. Consequently, a short but volatile session may receive a relatively large weight.

**Value**

A list with three components:

`realized_variance` A named numeric vector containing all supplied period variances. Non-overlapping periods appear first, followed by overlapping periods.

`variance_weights` A named numeric vector containing the normalized period weights  $w_h$ . It has the same names and order as `realized_variance` and sums to one, up to numerical precision.

`overlapping_weight` A numeric scalar equal to the sum of the weights belonging to `overlapping_variance`. It lies between zero and one.

**Interpretation**

The weights measure contributions to price variation, not proportions of trading time. A short but volatile period may therefore receive a larger weight than a longer period with little efficient-price variation.

In the final CWIS calculation, the overlapping-period weight is multiplied by the Hasbrouck information shares estimated while both markets are open. A non-overlapping-period weight is assigned entirely to the market that is open during that period.

The calculation assumes that all supplied values estimate the same variance concept and are expressed on a comparable scale. It also relies on the additivity of quadratic variation across disjoint intraday periods.

As noted by Dimpfl and Schweikert (2023), these weights do not adjust for possible differences in estimation precision, sampling error, or microstructure noise across periods. Their advantage is that they retain a direct interpretation as shares of total efficient-price variation. Hansen and Lunde (2005) discuss alternative approaches for combining variance estimates obtained from different portions of a trading day.

**Input requirements**

`overlapping_variance` and `non_overlapping_variance` must be named numeric vectors. Each element must contain one integrated-variance estimate for one trading period.

All names must be non-missing, non-empty, and unique across both vectors. The names are retained in the returned vectors and are used to determine which weights belong to overlapping periods.

All variance estimates must be finite and non-negative. Their combined sum must be strictly positive. The function does not verify how the estimates were obtained or whether they belong to the same trading day.

**References**

- Hansen, P. R. and Lunde, A. (2005). A realized variance for the whole day based on intermittent high-frequency data. *Journal of Financial Econometrics*, 3(4), 525–554. doi:10.1093/jfinec/nbi028
- Wang, J. and Yang, M. (2011). Housewives of Tokyo versus the gnomes of Zurich: Measuring price discovery in sequential markets. *Journal of Financial Markets*, 14(1), 82–108. doi:10.1016/j.finmar.2010.08.002
- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

**See Also**

`common_price()` for extracting the common efficient price; `realized_variance()` for overlapping-period variance; `realized_kernel()` for single-market-period variance; `his()` for Hasbrouck information shares; `aggregate_cwis()` for combining the variance weights with information shares; and `daily_cwis()` for the complete daily estimation procedure.

**Examples**

```
overlapping_variance <- c(
  overlap_pre = 1.2e-5,
  overlap_core = 4.5e-5,
  overlap_post_1 = 0.5e-5,
  overlap_post_2 = 0.8e-5
)

non_overlapping_variance <- c(
  futures_pre = 1.0e-5,
  spot_maintenance = 0.5e-5,
  futures_post = 1.5e-5
)

result <- combine_weights(
  overlapping_variance = overlapping_variance,
  non_overlapping_variance = non_overlapping_variance
)

result$realized_variance
result$variance_weights
result$overlapping_weight

sum(result$variance_weights)
```

---

common\_price

---

*Extract the common price from a VECM*


---

**Description**

Extracts a cumulative permanent common-price component from a fitted bivariate VECM using its common-trend representation. The function also returns the long-run impact matrix and the correlation between the two residual series used in the calculation.

**Usage**

```
common_price(vecm_fit, rank = 1L)
```

**Arguments**

|          |                                                                                                                                                                                                                 |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| vecm_fit | A fitted bivariate <code>ca.jo</code> object, normally returned by <code>vecm()</code> . Its variables must be ordered as futures followed by spot.                                                             |
| rank     | Integer cointegration rank passed to <code>urca::cajorls()</code> . The bivariate CWIS calculation requires <code>rank = 1</code> , corresponding to one cointegrating relation and one permanent common trend. |

### Details

The function is motivated by the "one security - many markets" framework of Hasbrouck (1995). Under this framework, an asset has a single latent efficient price even when it is traded in multiple markets. Observed prices may temporarily deviate from this fundamental value because of market-microstructure effects.

Let a trading day  $d$  consist of  $T$  equally spaced observations indexed by  $t$ . The day is divided into  $H$  disjoint intraday intervals,

$$S_h = \{t \mid t_{h-1}^\circ < t \leq t_h^\circ\}, \quad h = 1, \dots, H,$$

where  $t_h^\circ$  denotes the final observation in interval  $S_h$ , with  $t_0^\circ = 0$  and  $t_H^\circ = T$ . Together, the intervals  $S_1, \dots, S_H$  cover the entire trading day.

The latent efficient price is unobserved and is assumed to evolve as a random walk irrespective of how the trading day is partitioned:

$$m_{d,t} = m_{d,t-1} + e_{d,t},$$

where  $e_{d,t}$  is an independent mean-zero innovation. The innovations need not be identically distributed, allowing the variance of the efficient price to differ across intraday intervals.

When futures and spot markets trade simultaneously, their observed prices may temporarily differ because of market-microstructure effects. Under the law of one price, however, the two log-price series are expected to share a common long-run stochastic trend. Their joint long-run and short-run dynamics can therefore be represented by a bivariate VECM.

Let the bivariate VECM be written, for the transitory specification, as

$$\Delta p_t = \Pi p_{t-1} + \sum_{i=1}^{K-1} \Gamma_i \Delta p_{t-i} + d_t + u_t,$$

where  $p_t$  contains the futures and spot log prices,  $\Pi = \alpha\beta'$  is the long-run coefficient matrix,  $\beta$  contains the cointegrating vector,  $\alpha$  contains the adjustment coefficients,  $\Gamma_i$  contains the short-run coefficient matrices,  $d_t$  represents deterministic terms, and  $u_t$  denotes the reduced-form innovations.

According to Stock and Watson (1988), the VECM can equivalently be written in the common-trend representation

$$p_t = p_{t_1} + \Psi(1) \sum_{i=1}^t u_i + \Psi^*(L) u_t.$$

The long-run impact matrix is

$$\Psi(1) = C = \beta_\perp (\alpha'_\perp \Upsilon \beta_\perp)^{-1} \alpha'_\perp,$$

where

$$\Upsilon = I_2 - \sum_{i=1}^{K-1} \Gamma_i,$$

and  $I_2$  is the two-dimensional identity matrix. Under cointegration rank one,  $\beta_\perp$  and  $\alpha_\perp$  denote vectors orthogonal to  $\beta$  and  $\alpha$ , respectively. For a two-variable system, they are constructed as

$$\beta_{\perp} = \begin{pmatrix} -\beta_2 \\ \beta_1 \end{pmatrix}, \quad \alpha_{\perp} = \begin{pmatrix} -\alpha_2 \\ \alpha_1 \end{pmatrix}.$$

Here,  $p_{t_1}$  is a vector of initial values and  $\Psi^*(L)$  is a matrix polynomial in the lag operator  $L$ . The term  $C \sum_{i=1}^t u_i$  is the permanent common-trend component, whereas  $\Psi^*(L)u_t$  is a stationary transitory component.

The scalar efficient price of the asset is therefore

$$m_t = (\alpha'_{\perp} \Upsilon \beta_{\perp})^{-1} \alpha'_{\perp} \sum_{i=1}^t u_i,$$

whose innovations are serially uncorrelated by construction.

The function also reports the correlation between the two residual series stored in the `R0` slot of `vecm_fit`. This correlation is calculated as

$$\hat{\rho} = \text{cor}(R_{0,1}, R_{0,2}).$$

A high residual correlation indicates that contemporaneous innovations are difficult to attribute uniquely to either market. This is relevant for the width of the lower and upper Hasbrouck information-share bounds, as discussed by Hasbrouck (1995) and Dimpfl and Schweikert (2023).

## Value

A list with three components:

- `common_price`: a numeric vector of length `nrow(vecm_fit@R0)` containing the first component of the accumulated permanent common trend. The initial observed price level is not added;
- `residual_correlation`: the Pearson correlation between the two residual columns in `vecm_fit@R0`; a larger absolute correlation is generally associated with wider Hasbrouck bounds.
- `long_run_impact`: a numeric 2 by 2 matrix containing  $\hat{C}$ , the estimated long-run impact of the residuals on the permanent price component.

## Relation to the referenced methodology

Dimpfl and Schweikert (2023) use the common-trend representation to recover efficient-price innovations during overlapping trading periods. They then calculate realized variance from those innovations, avoiding an arbitrary choice between observed futures and spot returns. Dias and Schweikert (2022) provide supporting evidence for using common-trend returns when the same asset trades in fragmented markets.

The paper estimates a high-resolution VECM with HAR-structured lag coefficients. The present function instead receives the standard Johansen VECM fitted by `vecm()` and constructs  $\Upsilon$  from its unrestricted short-run coefficient matrices. It therefore implements the same common-trend principle but does not exactly reproduce the paper's HAR-VECM. See `vecm()` for further details on the VECM specification and its relation to the reference paper.

## Input requirements

`vecm_fit` must be a bivariate `ca.jo` object with the futures price ordered first and the spot price second. It should normally be created by `vecm()` so that the coefficient layout is compatible with the package's internal extraction functions.

The model must contain at least one differenced lag, meaning that the level VAR lag order used by `vecm()` must be at least two. The relevant residual and coefficient matrices must contain finite values.

For the bivariate CWIS framework, rank must be 1. This corresponds to two non-stationary price series connected by one cointegrating relationship and driven by one permanent common stochastic trend.

The scalar

$$\alpha'_{\perp} \Upsilon \beta_{\perp}$$

must be finite and sufficiently different from zero. Otherwise, the long-run impact matrix is singular and the function stops with an error.

## References

- Stock, J. H. and Watson, M. W. (1988). Testing for common trends. *Journal of the American Statistical Association*, 83(404), 1097–1107. doi:[10.2307/2290142](https://doi.org/10.2307/2290142)
- Gonzalo, J. and Granger, C. (1995). Estimation of common long-memory components in cointegrated systems. *Journal of Business & Economic Statistics*, 13(1), 27–35.
- Dias, G. F. and Schweikert, K. (2022). Integrated variance estimation for assets traded in multiple venues. *SSRN Working Paper*, 1–48. <https://ssrn.com/abstract=4253762>
- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:[10.1016/j.jbankfin.2023.106970](https://doi.org/10.1016/j.jbankfin.2023.106970)

## See Also

`vecm()` for fitting the bivariate Johansen VECM; `urca::cajorls()` for estimating the restricted VECM at a specified cointegration rank; `his()` for Hasbrouck information shares; `realized_variance()` for estimating variation from the common-price changes; `daily_cwis()` for the complete daily CWIS calculation.

## Examples

```
set.seed(123)

n <- 300
common_trend <- cumsum(rnorm(n, sd = 0.002))
stationary_spread <- as.numeric(
  stats::arima.sim(model = list(ar = 0.7), n = n, sd = 0.0005)
)

simulated_log_prices <- cbind(
  futures = log(5000) + common_trend + stationary_spread / 2,
  spot = log(5000) + common_trend - stationary_spread / 2
)

fitted_vecm <- vecm(
  log_prices = simulated_log_prices,
```

```

    K = 2,
    type = "trace",
    ecdet = "none",
    spec = "transitory"
  )

  common <- common_price(fitted_vecm, rank = 1L)

  names(common)
  head(common$common_price)
  common$residual_correlation
  common$long_run_impact

```

cwis

*Estimate 24-hour contribution-weighted information shares*

## Description

Estimates daily and aggregate contribution-weighted information shares (CWIS) for two markets with partially overlapping trading hours. The function combines Hasbrouck information shares from periods in which both markets trade with variance contributions from periods in which only one market trades.

## Usage

```

cwis(
  data,
  datetime_col,
  futures_col,
  spot_col,
  spot_multiplier = 10,
  tz,
  K = 10L,
  kernel_type = "ModifiedTukeyHanning",
  bandwidth_constant = NULL,
  align_by = "seconds",
  align_period = 1L,
  session_boundaries = .default_session_boundaries(),
  continue_on_error = TRUE
)

```

## Arguments

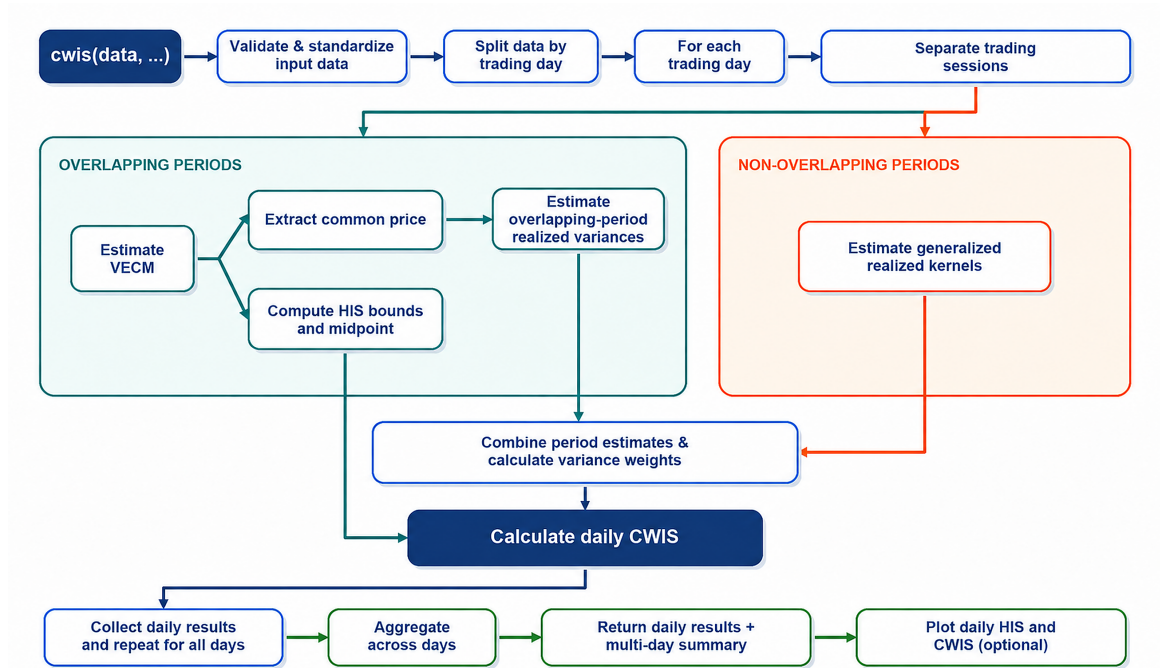
|              |                                                                                                                                                                          |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data         | A data frame containing one timestamp column and two synchronized numeric price columns representing the futures market and the corresponding spot, cash, or ETF market. |
| datetime_col | A single character string naming the timestamp column. Its values must be character or inherit from POSIXt.                                                              |
| futures_col  | A single character string naming the numeric futures-price column. This series is assumed to be observed during futures-only trading sessions.                           |

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| spot_col           | A single character string naming the numeric spot-, cash-, or ETF-price column. This series is assumed to be observed during the spot-only trading session.                                                                                                                                                                                                                                                                                                                                       |
| spot_multiplier    | A finite numeric scalar applied to the spot-price series during standardization, before logarithms are calculated. The multiplier should place the spot and futures prices on economically comparable scales. The default is 10, corresponding to the bundled S&P 500 futures and SPY example; use 1 when the supplied series are already on comparable scales.                                                                                                                                   |
| tz                 | A single character string specifying the IANA/Olson time zone used to parse timestamps, assign local trading dates, and interpret intraday session boundaries. Defaults to "America/New_York", consistent with the U.S. market application considered in the paper.                                                                                                                                                                                                                               |
| K                  | Integer lag order of the variables in the underlying level VAR passed to <code>urca::ca.jo()</code> . The corresponding VECM contains $K - 1$ lags of first differences. $K$ must be at least 2 for use with the downstream CWIS functions. The default is 10. Users may choose another value according to the sampling frequency, number of available observations, and lag-selection diagnostics.                                                                                               |
| kernel_type        | Character string specifying the kernel used by <code>highfrequency::rKernelCov()</code> to estimate realized kernels for non-overlapping trading sessions. Available kernel names can be inspected with <code>highfrequency::listAvailableKernels()</code> . The default, "ModifiedTukeyHanning", follows Dimpfl and Schweikert (2023).                                                                                                                                                           |
| bandwidth_constant | Optional positive numeric multiplier $c$ used to set the session-specific kernel parameter passed to <code>highfrequency::rKernelCov()</code> as $B = \lfloor c\sqrt{N} \rfloor$ , where $N$ is the number of rows in the session data. When NULL, $c = 2.3970$ is used for "ModifiedTukeyHanning" and $c = 3.5134$ for "Parzen". Other kernel types require an explicitly supplied positive value.                                                                                               |
| align_by           | Character string specifying the sampling unit in which <code>align_period</code> is expressed when aligning the non-overlapping session data. Supported values include "ticks", "seconds", "minutes", and "hours". Set to NULL to retain the original observation frequency, in which case <code>align_period</code> is ignored.                                                                                                                                                                  |
| align_period       | Positive numeric value specifying the number of <code>align_by</code> units in each sampling interval. It is ignored when <code>align_by</code> = NULL.                                                                                                                                                                                                                                                                                                                                           |
| session_boundaries | A strictly increasing named numeric vector defining the trading-session boundaries in seconds after local midnight. It must contain, in order, <code>overlap_open</code> , <code>core_open</code> , <code>core_close_exclusive</code> , <code>maintenance_start</code> , <code>overlap_resume</code> , and <code>overlap_close</code> . All values must be finite and lie strictly between 00:00:00 and 24:00:00. See <code>split_trading_sessions()</code> for the precise interval definitions. |
| continue_on_error  | Logical scalar controlling the treatment of errors raised during individual daily calculations. See the Failed trading days section.                                                                                                                                                                                                                                                                                                                                                              |

## Details

`cwis()` is the main user-level function of the package. It validates and standardizes the input data, divides the observations into trading days, calculates a CWIS estimate for each usable day, and aggregates the daily estimates and diagnostics.

The calculation follows the framework of Dimpfl and Schweikert (2023). The complete workflow is summarized below:



The calculation proceeds in four main stages:

1. `validate_and_standardize_input()` checks and standardizes the complete input data.
2. `split_trading_days()` divides the standardized observations into individual trading days.
3. `daily_cwis()` calculates the CWIS and diagnostics for each trading day.
4. `aggregate_cwis()` summarizes the daily results across trading days.

For each trading day, `daily_cwis()` divides the observations into overlapping and single-market sessions. It estimates the midpoint Hasbrouck information shares for overlapping trading and the variance contribution of each intraday period. During a single-market session, the open market is assigned an information share of one and the closed market an information share of zero. The resulting components are combined using normalized variance weights.

The daily futures and spot CWIS estimates sum to one, apart from numerical rounding.

Details of the session definitions, VECM, common efficient price, Hasbrouck information shares, variance estimators, and variance weights are documented in `split_trading_sessions()`, `vecm()`, `common_price()`, `his()`, `realized_variance()`, `realized_kernel()`, and `combine_weights()`.

## Value

An object of class `cwis_result`, represented by a list with the following components:

**daily** A data frame with one row per supplied trading day. It contains date, status, error, the futures and spot HIS midpoints, the futures and spot CWIS estimates, the total overlapping-period weight, the contemporaneous VECM residual correlation, the zero-return fractions, and the total and overlapping observation counts. Numerical results are NA for failed days.

**summary** A data frame reporting the mean and standard deviation of the successful daily HIS midpoints and CWIS estimates for both markets. Failed days are excluded.

**diagnostics** A data frame reporting the number of supplied, successful, and failed days, the mean and standard deviation of the overlapping-period weight, and the mean contemporaneous residual correlation.

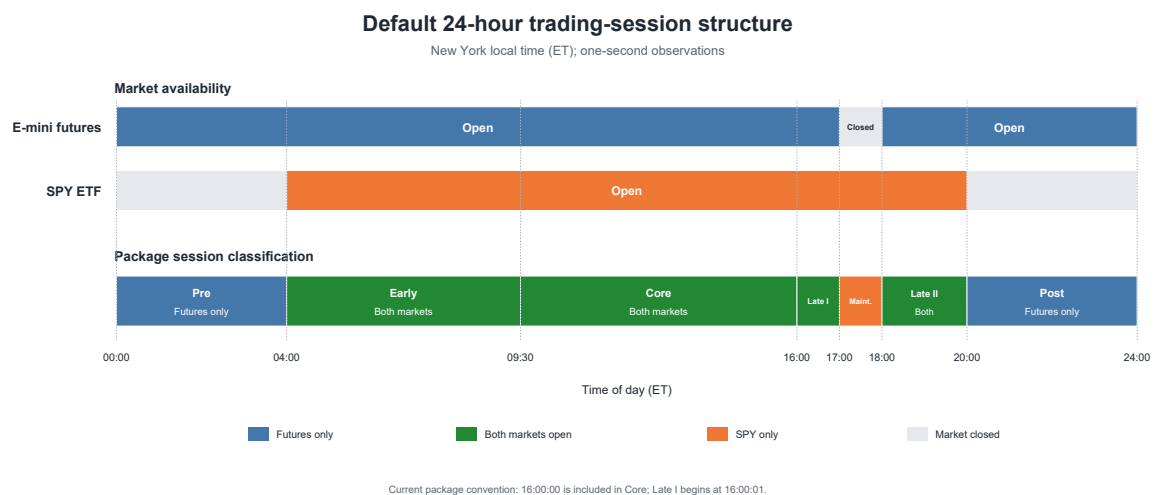
**variance\_weights** A data frame containing the mean and standard deviation of each period-specific variance weight across successful days. It is empty if no daily calculation succeeds.

**details** A named list containing the complete `cwis_day_result` object for every successful day. Failed entries contain only the trading date and error message.

**settings** A list recording the column mappings, scaling factor, time zone, VECM lag order, kernel and alignment controls, and session boundaries used in the calculation.

## Trading sessions

### Trading-session structure:



With the default `session_boundaries`, each calendar day in `tz` is divided into the following market configurations:

- 00:00–04:00: futures market only;
- 04:00–17:00: overlapping trading;
- 17:00–18:00: spot market only;
- 18:00–20:00: overlapping trading;
- 20:00–24:00: futures market only.

The overlapping observations are further divided into pre-core, core, and post-core subperiods for the variance calculation. However, a single VECM is estimated using the combined observations from all overlapping subperiods, thereby imposing common VECM parameters across these periods.

These default boundaries reproduce the futures and ETF trading schedule considered by Dimpfl and Schweikert (2023). For other assets or exchanges, users should specify boundaries that reflect the applicable trading schedule. See `split_trading_sessions()` for the precise interval definitions.

## Input requirements

data must be a data frame containing one timestamp column and two numeric price columns. The column names are supplied as character strings through `datetime_col`, `futures_col`, and `spot_col`.

The timestamp column must be character or inherit from `POSIXt`. Character timestamps are interpreted in `tz`. Missing or unparseable timestamps are removed before the daily calculations. The remaining observations are sorted chronologically, but duplicate timestamps cause the complete calculation to stop.

Prices must be finite and strictly positive after applying `spot_multiplier`. Missing prices are not imputed. Although individual missing prices can pass the initial standardization, the affected trading day will generally fail when complete prices are required by the VECM or realized-kernel calculation.

The two price series should represent synchronized observations of the same underlying asset, or assets linked by a stable law-of-one-price relationship. The current session logic assumes that the series identified by `futures_col` trades during the futures-only periods and the series identified by `spot_col` trades during the spot-only period.

Every session required by `daily_cwis()` must contain at least two observations. The combined overlapping period must contain more than  $K + 2$  complete observations, although substantially more observations are normally required for stable VECM estimation.

## Time zone and sampling

`tz` controls timestamp parsing, trading-day assignment, and the interpretation of the intraday session boundaries. It should be a valid Olson time-zone name, such as "America/New\_York", "America/Chicago", "Europe/London", "Europe/Paris", or "UTC".

Trading days run from local midnight to the following local midnight in `tz`. Users should check the treatment of daylight-saving transitions, overnight trading conventions, and exchange holidays before applying the default boundaries to another market.

The defaults `align_by = "seconds"` and `align_period = 1L` align single-market observations to one-second intervals. They do not reproduce the 10-millisecond sampling frequency used in the empirical application of Dimpfl and Schweikert (2023). Users may change the alignment settings when supported by `highfrequency::rKernelCov()` and appropriate for their data.

## Choice of the VECM lag order

In this implementation,  $K$  is passed directly to `urca::ca.jo()` and denotes the lag order of the variables in the underlying level VAR. The corresponding standard Johansen VECM contains  $K - 1$  differenced lags. Thus,  $K = 10L$  produces nine short-run lag matrices.

Users may select another value of  $K$ , subject to  $K \geq 2$  and the availability of sufficient observations. Lag selection should consider the sampling frequency, remaining serial dependence, and numerical stability.

The reference paper uses a different specification of the short-run dynamics. At a sampling frequency of 10 milliseconds, Dimpfl and Schweikert (2023) set  $\delta = 4$ , with time-scale boundaries  $k_1 = 1$ ,  $k_2 = 10$ ,  $k_3 = 100$ , and  $k_4 = 1000$ . The resulting four HAR coefficient blocks cover lagged price changes over a maximum horizon of ten seconds. In contrast, the present function estimates  $K - 1$  separate short-run coefficient matrices using the standard Johansen VECM. It therefore implements the VECM component of the CWIS framework but does not exactly reproduce the paper's high-resolution HAR-VECM specification.

### Failed trading days

If `continue_on_error = TRUE`, an error arising during a particular daily calculation is captured and the date is retained in the output with `status = "failed"`, the corresponding error message, and missing numerical estimates. Calculations then continue for the remaining dates.

If `continue_on_error = FALSE`, the first daily error stops the entire calculation.

Errors occurring before the data are divided into days, including invalid column names, invalid timestamp types, duplicate timestamps, and invalid observed prices, are not handled by `continue_on_error` and always stop the function.

### References

- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970
- Hasbrouck, J. (1995). One security, many markets: Determining the contributions to price discovery. *The Journal of Finance*, 50(4), 1175–1199. doi:10.2307/2329348
- Dias, G. F. and Schweikert, K. (2022). Integrated variance estimation for assets traded in multiple venues. *SSRN Working Paper*, 1–48. <https://ssrn.com/abstract=4253762>
- Wang, J. and Yang, M. (2011). Housewives of Tokyo versus the gnomes of Zurich: Measuring price discovery in sequential markets. *Journal of Financial Markets*, 14(1), 82–108. doi:10.1016/j.finmar.2010.08.002
- Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., and Shephard, N. (2008). Designing realized kernels to measure the ex post variation of equity prices in the presence of noise. *Econometrica*, 76(6), 1481–1536. doi:10.3982/ECTA6495
- Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., and Shephard, N. (2009). Realized kernels in practice: Trades and quotes. *The Econometrics Journal*, 12(3), 1–32. doi:10.1111/j.1368-423X.2008.00275.x
- Hasbrouck, J. (2021). Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 395–430. doi:10.1093/jfinec/nbz027
- Buccheri, G., Bormetti, G., Corsi, F., and Lillo, F. (2021). Comment on: Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 439–451. doi:10.1093/jfinec/nbz008

### See Also

`daily_cwis()` for the complete single-day calculation; `validate_and_standardize_input()` for input validation; `split_trading_sessions()` for session definitions; `vecm()` and `common_price()` for estimation of the common efficient price; `his()` for Hasbrouck information shares; `realized_variance()` and `realized_kernel()` for period variances; `combine_weights()` for variance weights; `aggregate_cwis()` for aggregation across days; and `plot.cwis_result()` for plotting daily results.

### Examples

```
set.seed(123)

tz <- "America/New_York"
datetime <- seq(
  from = as.POSIXct("2024-01-03 00:00:00", tz = tz),
  to = as.POSIXct("2024-01-03 23:59:00", tz = tz),
  by = "1 min"
)

n <- length(datetime)
```

```

common_trend <- cumsum(stats::rnorm(n, sd = 0.0002))
spread <- as.numeric(stats::arima.sim(
  model = list(ar = 0.8),
  n = n,
  sd = 0.0001
))

example_data <- data.frame(
  datetime = datetime,
  futures_price = exp(log(5000) + common_trend + spread / 2),
  spot_price = exp(log(5000) + common_trend - spread / 2) / 10
)

result <- cwis(
  data = example_data,
  datetime_col = "datetime",
  futures_col = "futures_price",
  spot_col = "spot_price",
  spot_multiplier = 10,
  tz = tz,
  K = 2L,
  align_by = "minutes",
  align_period = 1L,
  continue_on_error = FALSE
)

print(result)
result$daily
result$summary
result$diagnostics

```

---

daily\_cwis

---

*Calculate CWIS for one trading day*


---

## Description

Calculates the contribution-weighted information share (CWIS) for a futures-spot pair over one trading day with partially overlapping trading hours.

The function combines Hasbrouck information shares from overlapping trading periods with realized variances from non-overlapping periods.

## Usage

```

daily_cwis(
  day_data,
  K = 10L,
  kernel_type = "ModifiedTukeyHanning",
  bandwidth_constant = NULL,
  align_by = "seconds",
  align_period = 1L,
  session_boundaries = .default_session_boundaries()
)

```

## Arguments

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| day_data           | A standardized data frame containing exactly one trading day and the columns <code>datetime</code> , <code>trading_day</code> , <code>futures</code> , and <code>spot</code> .                                                                                                                                                                                                                                                                                                                    |
| K                  | Integer lag order of the variables in the underlying level VAR passed to <code>urca::ca.jo()</code> . The corresponding VECM contains $K - 1$ lags of first differences. $K$ must be at least 2 for use with the downstream CWIS functions. The default is 10. Users may choose another value according to the sampling frequency, number of available observations, and lag-selection diagnostics.                                                                                               |
| kernel_type        | Character string specifying the kernel used by <code>highfrequency::rKernelCov()</code> to estimate realized kernels for non-overlapping trading sessions. Available kernel names can be inspected with <code>highfrequency::listAvailableKernels()</code> . The default, "ModifiedTukeyHanning", follows Dimpfl and Schweikert (2023).                                                                                                                                                           |
| bandwidth_constant | Optional positive numeric multiplier $c$ used to set the session-specific kernel parameter passed to <code>highfrequency::rKernelCov()</code> as $B = \lfloor c\sqrt{N} \rfloor$ , where $N$ is the number of rows in the session data. When NULL, $c = 2.3970$ is used for "ModifiedTukeyHanning" and $c = 3.5134$ for "Parzen". Other kernel types require an explicitly supplied positive value.                                                                                               |
| align_by           | Character string specifying the sampling unit in which <code>align_period</code> is expressed when aligning the non-overlapping session data. Supported values include "ticks", "seconds", "minutes", and "hours". Set to NULL to retain the original observation frequency, in which case <code>align_period</code> is ignored.                                                                                                                                                                  |
| align_period       | Positive numeric value specifying the number of <code>align_by</code> units in each sampling interval. It is ignored when <code>align_by</code> = NULL.                                                                                                                                                                                                                                                                                                                                           |
| session_boundaries | A strictly increasing named numeric vector defining the trading-session boundaries in seconds after local midnight. It must contain, in order, <code>overlap_open</code> , <code>core_open</code> , <code>core_close_exclusive</code> , <code>maintenance_start</code> , <code>overlap_resume</code> , and <code>overlap_close</code> . All values must be finite and lie strictly between 00:00:00 and 24:00:00. See <code>split_trading_sessions()</code> for the precise interval definitions. |

## Details

`day_data` is assumed to have been validated and standardized by `validate_and_standardize_input()` and restricted to a single trading day by `split_trading_days()`. In the complete package workflow, these preprocessing steps are performed by `cwis()`. The daily calculation then proceeds as follows:

1. `split_trading_sessions()` divides the observations into overlapping and single-market sessions. See that function for the exact session definitions.
2. The overlapping observations are pooled, and `vecm()` estimates the bivariate VECM under both possible market orderings.
3. `his()` is applied to both fitted VECMs. The two ordering-specific estimates provide the lower and upper Hasbrouck bounds for each market, and their average is used as the overlapping-period midpoint estimate.
4. `common_price()` extracts the common efficient price. Its returns are assigned to the pre-core, core, and combined post-core overlapping subperiods, for which `realized_variance()` estimates price variation.
5. `realized_kernel()` estimates price variation during each single-market session.

6. `combine_weights()` converts the session-specific variance estimates into normalized weights and calculates the total overlapping weight.
7. Finally, following Dimpfl and Schweikert (2023), the function calculates the daily CWIS for both markets:

$$\begin{pmatrix} S_{cw,1}^* \\ S_{cw,2}^* \end{pmatrix} = w_{\text{overlap}} \begin{pmatrix} S_1^* \\ S_2^* \end{pmatrix} + w_{\text{market1}} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + w_{\text{market2}} \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Here,  $S_j^*$  is the midpoint Hasbrouck information share of market  $j$  during overlapping trading. The term  $w_{\text{overlap}}$  is the total weight of the overlapping subperiods. The terms  $w_{\text{market1}}$  and  $w_{\text{market2}}$  are the total weights of the periods in which only market 1 or only market 2 is open. In this package, market 1 is the futures market and market 2 is the spot market. Therefore,  $w_{\text{market1}}$  combines the weights of `futures_pre` and `futures_post`, while  $w_{\text{market2}}$  is the weight of `spot_maintenance`.

When the weights are normalized, the two daily CWIS estimates sum to one. If  $w_{\text{overlap}} = 1$ , they equal the midpoint Hasbrouck information shares.

Each referenced function documents the corresponding calculation and input requirements in detail.

## Value

An object of class `cwis_day_result`, implemented as a list with the following components:

- `date` The single trading date represented by `day_data`.
- `his_lower` A named numeric vector containing the lower Hasbrouck information-share bound for futures and spot.
- `his_upper` A named numeric vector containing the upper Hasbrouck information-share bound for futures and spot.
- `his_midpoint` A named numeric vector containing the midpoint of the lower and upper Hasbrouck bounds.
- `residual_correlation` The Pearson correlation between the two residual columns in `vecm_fit@R0`; A high correlation is generally associated with wider Hasbrouck bounds.
- `zero_return_fraction` A named numeric vector giving the proportion of zero log returns in the pooled overlapping sample for futures and spot.
- `realized_variance` A named numeric vector containing variance estimates for `futures_pre`, `spot_maintenance`, `futures_post`, `overlap_pre`, `overlap_core`, and `overlap_post`.
- `variance_weights` The corresponding normalized period weights. These sum to one up to numerical precision.
- `overlapping_weight` The total weight assigned to `overlap_pre`, `overlap_core`, and `overlap_post`.
- `cwis` A named numeric vector containing the daily CWIS for futures and spot.
- `n_observations` The total number of rows in `day_data`.
- `n_overlap` The total number of observations in the four overlapping component sessions.

## Relationship to the reference paper

Dimpfl and Schweikert (2023), following Hasbrouck (2021) and Buccheri et al. (2021), use a high-resolution VECM with HAR-structured lag coefficients. At a 10-millisecond sampling frequency, their specification uses four HAR time-scale blocks to cover lagged price changes over a maximum horizon of ten seconds.

The present function instead calls `vecm()`, which estimates the standard Johansen VECM implemented by `urca::ca.jo()`. Therefore, this function implements the CWIS framework and its variance-weighting principle but is not an exact replication of the paper's high-resolution HAR-VECM. See `vecm()` for further details on the VECM specification and its relation to the reference paper.

### Input requirements

`day_data` must be a standardized data frame containing the columns `datetime`, `trading_day`, `futures`, and `spot`. It should normally be obtained from `validate_and_standardize_input()` and must contain exactly one trading day.

Timestamps must be chronologically ordered and must be interpretable in the time zone stored in the "tz" attribute. Session membership is determined from local clock time using `session_boundaries`.

Each of the seven required component sessions must contain at least two observations. Both price series must be available, finite, and strictly positive during overlapping sessions. The price of the open market must be available, finite, and strictly positive during each non-overlapping session.

The pooled overlapping sample must contain more than  $K + 2$  complete observations, and both overlapping price series must vary. Missing values are not imputed.

### References

Hasbrouck, J. (2021). Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 395–430. doi:10.1093/jjfinec/nbz027

Bucheri, G., Bormetti, G., Corsi, F., and Lillo, F. (2021). Comment on: Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 439–451. doi:10.1093/jjfinec/nbz008

Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

### See Also

`split_trading_sessions()` for constructing the trading sessions; `vecm()` for estimating the overlapping-period VECM; `his()` for Hasbrouck information-share bounds; `common_price()` for extracting the common efficient price; `realized_variance()` for overlapping-period variance; `realized_kernel()` for single-market variance; `combine_weights()` for variance weights; and `aggregate_cwis()` for aggregating results across trading days.

### Examples

```
set.seed(123)

n <- 420L
datetime <- as.POSIXct(
  "2026-01-05 00:00:00",
  tz = "America/New_York"
) + 0:(n - 1L)

common_trend <- cumsum(rnorm(n, sd = 0.0002))
futures <- 5000 * exp(common_trend + rnorm(n, sd = 0.00005))
spot <- 5000 * exp(common_trend + rnorm(n, sd = 0.00007))

# Futures are closed during the spot-only session.
futures[241:300] <- NA_real_
```

```
# Spot is closed during the futures-only sessions.
spot[c(1:60, 361:420)] <- NA_real_

day_data <- data.frame(
  datetime = datetime,
  trading_day = as.Date("2026-01-05"),
  futures = futures,
  spot = spot
)
attr(day_data, "tz") <- "America/New_York"

example_boundaries <- c(
  overlap_open = 60,
  core_open = 120,
  core_close_exclusive = 180,
  maintenance_start = 240,
  overlap_resume = 300,
  overlap_close = 360
)

result <- daily_cwis(
  day_data = day_data,
  K = 2L,
  session_boundaries = example_boundaries
)

result$cwis
result$his_midpoint
result$variance_weights
result$overlapping_weight
```

---

```
example_sp500_futures_spot_oneweek_24h_1sec
```

*One week of one-second S&P 500 futures and spot prices*

---

## Description

An example data set containing synchronized S&P 500 futures and spot-price observations at one-second frequency over one week. The timestamps represent New York local time.

## Usage

```
example_sp500_futures_spot_oneweek_24h_1sec
```

## Format

A `data.table` and `data.frame` with 590400 rows and 5 variables:

**date** A character string giving the calendar date in "YYYY-MM-DD" format.

**time** A character string giving the New York local clock time in "HH:MM:SS" format.

**datetime** A character string combining date and time in "YYYY-MM-DD HH:MM:SS" format.

**V1** A numeric S&P 500 futures-price series.

**V2** A numeric S&P 500 spot-index series before application of the spot-price multiplier.

## Source

Data provided by the thesis supervisor for thesis research and package testing.

---

|     |                                                                   |
|-----|-------------------------------------------------------------------|
| his | <i>Compute Hasbrouck information shares for one VECM ordering</i> |
|-----|-------------------------------------------------------------------|

---

## Description

Computes Hasbrouck information shares for futures and spot markets from a fitted bivariate VECM and a specified ordering of the two price series.

## Usage

```
his(vecm_fit)
```

## Arguments

|          |                                                                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| vecm_fit | A fitted bivariate <code>ca.jo</code> object, normally returned by <code>vecm()</code> . Its variables must be ordered as futures followed by spot. |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

## Details

When two markets trade the same underlying asset, their observed prices may differ temporarily but are expected to share one permanent common stochastic trend. Hasbrouck's (1995) information share measures the proportion of the innovation variance of this common trend attributable to each market.

These common long-run and market-specific short-run dynamics can be represented by a cointegrated vector error-correction model (VECM). For the default transitory specification, the model estimated by `urca::ca.jo()` can be summarized as

$$\Delta p_t = \Pi p_{t-1} + \sum_{i=1}^{K-1} \Gamma_i \Delta p_{t-i} + d_t + u_t,$$

where  $p_t$  contains the futures and spot log prices,  $\Pi = \alpha\beta'$  is the long-run coefficient matrix,  $\beta$  is the cointegrating vector,  $\alpha$  contains the adjustment coefficients,  $\Gamma_i$  contains the short-run coefficient matrices,  $d_t$  represents the deterministic terms, and  $u_t$  is a vector of reduced-form white-noise innovations satisfying  $E(u_t) = 0$  and

$$\Omega = E(u_t u_t') = \text{Var}(u_t).$$

To allocate the contemporaneous covariance among the markets, the innovation covariance matrix is factorized as

$$\Omega = FF',$$

where  $F$  is a lower-triangular Cholesky factor. In the implementation, this factor is calculated from the covariance matrix stored in the `DELTA` slot of `vecm_fit` using `F <- t(chol(vecm_fit@DELTA))`.

Assuming that the cointegrated system is driven by one common stochastic trend, let  $\psi$  denote a row of the long-run impact matrix  $\Psi(1)$ . The variance of the common-trend innovation is then  $\psi\Omega\psi'$ .

The Hasbrouck information share of market  $j$  is

$$S_j = \frac{[\psi F]_j^2}{\psi \Omega \psi'}, \quad j = 1, 2.$$

The numerator is the squared contribution of the orthogonalized innovation associated with market  $j$ . The denominator is the total innovation variance of the permanent common-price component. Thus,  $S_j$  measures the proportion of permanent price innovation attributed to market  $j$  under the selected ordering.

### Value

A named numeric vector of length two with components:

- futures: the Hasbrouck information share assigned to the first variable under the current Cholesky ordering;
- spot: the Hasbrouck information share assigned to the second variable under the current Cholesky ordering.

When the decomposition is well-defined, both values are non-negative and sum to one. The returned values represent one ordering-specific allocation; they are not, by themselves, complete lower and upper bounds.

### Ordering and information-share bounds

When the market innovations are contemporaneously correlated, their joint covariance cannot be allocated uniquely to an individual market. The Cholesky decomposition resolves this covariance sequentially and therefore depends on the ordering of the markets.

Following Hasbrouck (1995), placing a market first in the ordering maximizes the information share assigned to that market and provides its upper bound. Placing the same market last minimizes its assigned information share and provides its lower bound. For two markets, both possible orderings are therefore evaluated, giving

$$S_j^L \leq S_j \leq S_j^U.$$

The bounds may coincide when the innovations are uncorrelated. A wider interval between  $S_j^L$  and  $S_j^U$  indicates greater uncertainty about how the contemporaneous covariance should be assigned.

The midpoint of the lower and upper bounds is commonly reported as a single information-share estimate:

$$S_j^* = \frac{S_j^L + S_j^U}{2}.$$

This function calculates information shares for only the ordering stored in `vecm_fit`. The reversed ordering must be estimated separately to obtain the other bound. `daily_cwis()` performs both ordering-specific calculations and uses  $S_j^*$  as the overlapping-period HIS estimate, following Dimpfl and Schweikert (2023).

High contemporaneous innovation correlation can produce wide bounds and make their midpoint less informative. Using a sufficiently high sampling frequency may reduce this correlation, although the sampling frequency should also be selected with regard to market-microstructure noise.

### Relation to the referenced methodology

The information-share decomposition follows Hasbrouck (1995). Dimpfl and Schweikert (2023) apply it to the periods in which futures and ETF markets trade simultaneously and use the midpoint of the two ordering-based bounds in their contribution-weighted information share.

The paper obtains the required long-run quantities from a high-resolution HAR-VECM following Hasbrouck (2021). This function instead extracts them from the standard Johansen VECM returned by `vecm()`. It therefore applies the Hasbrouck decomposition to the package's VECM implementation but does not reproduce the paper's HAR-VECM exactly. See `vecm()` for further details on the VECM specification and its relation to the reference paper.

### Input requirements

`vecm_fit` must be a fitted bivariate `ca.jo` object whose first variable is the futures log price and whose second variable is the spot log price. It should normally be returned by `vecm()`.

The model must be suitable for a rank-one representation and contain at least one differenced lag. The package's downstream methodology is designed for the default transitory VECM specification.

The covariance matrix stored in `vecm_fit@DELTA` must be finite, symmetric, and positive definite so that its Cholesky decomposition exists. The scalar

$$\alpha'_{\perp} \Upsilon \beta_{\perp}$$

must be finite and sufficiently different from zero. The total common-trend innovation variance must also be strictly positive. Otherwise, the function stops with an error.

### References

- Stock, J. H. and Watson, M. W. (1988). Testing for common trends. *Journal of the American Statistical Association*, 83(404), 1097–1107. doi:10.2307/2290142
- Hasbrouck, J. (1995). One security, many markets: Determining the contributions to price discovery. *The Journal of Finance*, 50(4), 1175–1199. doi:10.2307/2329348
- Putniņš, T. J. (2013). What do price discovery metrics really measure? *Journal of Empirical Finance*, 23, 68–83. doi:10.1016/j.jempfin.2013.05.004
- Hasbrouck, J. (2021). Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 395–430. doi:10.1093/jfinec/nbz027
- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

### See Also

`vecm()` for estimating the bivariate VECM; `common_price()` for extracting the permanent common-price component; `daily_cwis()` for computing both Cholesky orderings, the information-share bounds, and their midpoint; `combine_weights()` for combining the midpoint with period-specific variance weights.

### Examples

```
set.seed(123)

n <- 300
common_trend <- cumsum(rnorm(n, sd = 0.002))
stationary_spread <- as.numeric(
```

```

stats::arima.sim(model = list(ar = 0.7), n = n, sd = 0.0005)
)

log_prices <- cbind(
  futures = log(5000) + common_trend + stationary_spread / 2,
  spot = log(5000) + common_trend - stationary_spread / 2
)

# Information shares for the futures-first ordering
fit_futures_first <- vecm(
  log_prices = log_prices,
  K = 2,
  type = "trace",
  ecdet = "none",
  spec = "transitory"
)

shares_futures_first <- his(fit_futures_first)
shares_futures_first
sum(shares_futures_first)

# Repeat the calculation after reversing the market order
fit_spot_first <- vecm(
  log_prices = log_prices[, c("spot", "futures")],
  K = 2,
  type = "trace",
  ecdet = "none",
  spec = "transitory"
)

shares_spot_first_position <- his(fit_spot_first)

# Restore the reversed result to the original futures-spot order
shares_spot_first <- stats::setNames(
  rev(unname(shares_spot_first_position)),
  c("futures", "spot")
)

bounds <- cbind(
  lower = pmin(shares_futures_first, shares_spot_first),
  upper = pmax(shares_futures_first, shares_spot_first)
)

midpoint <- rowMeans(bounds)

bounds
midpoint

```

---

plot.cwis\_result

---

*Plot daily CWIS and Hasbrouck information shares*


---

### Description

Draws the daily Hasbrouck information-share midpoint and the daily contribution-weighted information share for either the futures or spot market. Only days with status == "ok" are plotted. A

horizontal benchmark is added at 0.5 by default, and the horizontal axis displays the dates of successful trading days. When there are many successful days, a subset of evenly spaced date labels is shown to keep the axis readable.

### Usage

```
## S3 method for class 'cwis_result'
plot(
  x,
  y = NULL,
  market = c("futures", "spot"),
  his_colour = "blue",
  cwis_colour = "black",
  benchmark = 0.5,
  benchmark_colour = "grey50",
  line_width = 1.5,
  ylim = c(0, 1),
  xlab = "Trading date",
  ylab = "",
  main = NULL,
  date_format = "%Y-%m-%d",
  max_date_labels = 20L,
  show_legend = TRUE,
  legend_position = "topright",
  mar = c(8, 3, 0.5, 0.5),
  ...
)
```

### Arguments

|                  |                                                                                                       |
|------------------|-------------------------------------------------------------------------------------------------------|
| x                | A <code>cwis_result</code> object returned by <code>cwis()</code> .                                   |
| y                | Unused. This argument is included for compatibility with the <code>graphics::plot()</code> generic.   |
| market           | Character string selecting the market to plot. Must be either "futures" or "spot".                    |
| his_colour       | Colour used for the daily Hasbrouck information-share midpoint.                                       |
| cwis_colour      | Colour used for the daily contribution-weighted information share.                                    |
| benchmark        | Numeric scalar giving the horizontal reference level.                                                 |
| benchmark_colour | Colour used for the benchmark line.                                                                   |
| line_width       | Numeric line width used for the HIS and CWIS series.                                                  |
| ylim             | Numeric vector of length two giving the vertical plotting range.                                      |
| xlab, ylab       | Character strings giving the horizontal and vertical axis labels.                                     |
| main             | Optional plot title.                                                                                  |
| date_format      | Character string passed to <code>base::format.Date()</code> to control the appearance of date labels. |
| max_date_labels  | Maximum number of date labels displayed on the horizontal axis.                                       |
| show_legend      | Logical scalar indicating whether a legend is displayed.                                              |

**legend\_position** Position passed to `graphics::legend()`.  
**mar** Numeric vector of four plot margins passed to `graphics::par()`.  
**...** Additional graphical parameters passed to `graphics::plot()`.

### Value

Invisibly returns a data frame containing the plotted dates, daily status values, HIS values, and CWIS values.

### Examples

```
## Not run:
fit <- cwis(
  data = example_sp500_futures_spot_oneweek_24h_1sec,
  datetime_col = "datetime",
  futures_col = "V1",
  spot_col = "V2",
  tz = "America/New_York"
)

plot(fit, market = "futures")
plot(fit, market = "spot")

## End(Not run)
```

---

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| realized_kernel | <i>Estimate realized-kernel variance for a single-market session</i> |
|-----------------|----------------------------------------------------------------------|

---

### Description

Estimates the integrated variance of one price series during a trading session in which only one market is open. A generalized realized-kernel estimator is used to reduce the effect of market-microstructure noise in high-frequency returns.

### Usage

```
realized_kernel(
  session_data,
  price_col,
  kernel_type = "ModifiedTukeyHanning",
  bandwidth_constant = NULL,
  align_by = "seconds",
  align_period = 1L
)
```

### Arguments

**session\_data** A standardized data frame for one single-market trading session. It must contain a datetime column and the price series selected by `price_col`.

|                    |                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| price_col          | Character string identifying the price series to use. Must be either "futures" or "spot".                                                                                                                                                                                                                                                                                                           |
| kernel_type        | Character string specifying the kernel used by <code>highfrequency::rKernelCov()</code> to estimate realized kernels for non-overlapping trading sessions. Available kernel names can be inspected with <code>highfrequency::listAvailableKernels()</code> . The default, "ModifiedTukeyHanning", follows Dimpfl and Schweikert (2023).                                                             |
| bandwidth_constant | Optional positive numeric multiplier $c$ used to set the session-specific kernel parameter passed to <code>highfrequency::rKernelCov()</code> as $B = \lfloor c\sqrt{N} \rfloor$ , where $N$ is the number of rows in the session data. When NULL, $c = 2.3970$ is used for "ModifiedTukeyHanning" and $c = 3.5134$ for "Parzen". Other kernel types require an explicitly supplied positive value. |
| align_by           | Character string specifying the sampling unit in which align_period is expressed when aligning the non-overlapping session data. Supported values include "ticks", "seconds", "minutes", and "hours". Set to NULL to retain the original observation frequency, in which case align_period is ignored.                                                                                              |
| align_period       | Positive numeric value specifying the number of align_by units in each sampling interval. It is ignored when align_by = NULL.                                                                                                                                                                                                                                                                       |

## Details

During a non-overlapping trading period, only one market price is observed. Following Wang and Yang (2011), the open market is assigned the price discovery occurring during that period. Its contribution to the daily CWIS is measured by the variation in the efficient price generated while that market is open.

At very high sampling frequencies, observed returns may be serially correlated because of bid-ask bounce, price discreteness, and other microstructure effects. The ordinary realized-variance estimator may therefore be distorted. Dimpfl and Schweikert (2023) address this problem in non-overlapping sessions using the realized-kernel estimator developed by Barndorff-Nielsen et al. (2008).

For a session containing  $n$  intraday returns, the estimator is

$$\widehat{IV}_K^{(n)} = \sum_{i=1}^n r_{i,n}^2 + \sum_{b=1}^B k\left(\frac{b-1}{B}\right) (\eta_b + \eta_{-b}),$$

where

$$\eta_b = \sum_{i=1}^{n-b} r_{i,n} r_{i+b,n}.$$

Here,  $r_{i,n}$  is the  $i$ -th intraday return,  $n$  is the number of returns,  $B$  is the kernel bandwidth,  $k(\cdot)$  is the selected kernel-weight function, and  $\eta_b$  and  $\eta_{-b}$  are return cross-products at positive and negative lag  $b$ . The weighted autocovariance terms correct for serial dependence induced by market-microstructure noise.

The selected price series is converted to an `xts::xts()` object. `highfrequency::rKernelCov()` optionally aligns the prices to the requested sampling frequency, constructs returns internally, and applies the selected kernel. Only the first univariate variance estimate is returned.

The resulting estimate is subsequently used as the variance contribution of the corresponding single-market session when computing the period weights and the daily contribution-weighted information share.

## Value

An unnamed numeric scalar containing the realized-kernel estimate of integrated variance for the selected price series and session. The result is expressed in squared-return units; it is neither annualized nor converted to volatility by taking its square root.

## Sampling and bandwidth

`align_by` and `align_period` determine the sampling frequency before returns are calculated. For example, `align_by = "seconds"` and `align_period = 1` produce one-second sampling, whereas `align_by = "minutes"` and `align_period = 5` produce five-minute sampling.

Supported time units are "ticks", "secs", "seconds", "mins", "minutes", and "hours". The underlying function does not provide a "milliseconds" option. To retain observations that have already been sampled at a sub-second frequency, such as 10 milliseconds, set `align_by = NULL`; `align_period` is then ignored.

Let  $N$  denote the number of rows in `session_data`. This wrapper selects the kernel parameter as

$$B = \left\lfloor c\sqrt{N} \right\rfloor,$$

where  $c$  is `bandwidth_constant`. If `bandwidth_constant = NULL`,  $c = 2.3970$  is used for the modified Tukey-Hanning kernel and  $c = 3.5134$  for the Parzen kernel. For any other kernel, `bandwidth_constant` must be supplied explicitly.

Dimpfl and Schweikert (2023) use the modified Tukey-Hanning kernel with the bandwidth selected according to the rule-of-thumb proposed by Barndorff-Nielsen et al. (2009).

## Input requirements

`session_data` must be a data frame containing datetime and the column selected by `price_col`. It should represent one standardized single-market session, normally obtained from `split_trading_sessions()`.

The datetime column must contain chronologically ordered timestamps suitable for constructing an xts index. Timestamps should be unique and belong to one trading day and one trading session.

The selected price column must contain at least two numeric observations. All selected prices must be finite and strictly positive; missing values are not permitted. More observations may be required for a meaningful kernel estimate, particularly when a large bandwidth or coarse sampling interval is selected.

## References

- Hansen, P. R. and Lunde, A. (2006). Realized variance and market microstructure noise. *Journal of Business & Economic Statistics*, 24(2), 127–161. doi:10.1198/073500106000000071
- Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., and Shephard, N. (2008). Designing realized kernels to measure the ex post variation of equity prices in the presence of noise. *Econometrica*, 76(6), 1481–1536. doi:10.3982/ECTA6495
- Barndorff-Nielsen, O. E., Hansen, P. R., Lunde, A., and Shephard, N. (2009). Realized kernels in practice: Trades and quotes. *The Econometrics Journal*, 12(3), 1–32. doi:10.1111/j.1368-423X.2008.00275.x
- Wang, J. and Yang, M. (2011). Housewives of Tokyo versus the gnomes of Zurich: Measuring price discovery in sequential markets. *Journal of Financial Markets*, 14(1), 82–108. doi:10.1016/j.finmar.2010.08.002
- Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

**See Also**

`highfrequency::rKernelCov()` for the underlying estimator; `highfrequency::listAvailableKernels()` for the supported kernels; `realized_variance()` for ordinary realized variance; `split_trading_sessions()` for constructing session data; `combine_weights()` for converting session variances into contribution weights; and `daily_cwis()` for the complete daily calculation.

**Examples**

```
set.seed(123)

n <- 600L
session <- data.frame(
  datetime = as.POSIXct(
    "2026-01-05 00:00:00",
    tz = "America/New_York"
  ) + seq_len(n) - 1,
  trading_day = as.Date("2026-01-05"),
  futures = 5000 * exp(cumsum(rnorm(n, sd = 0.0001))),
  spot = NA_real_
)

rk <- realized_kernel(
  session_data = session,
  price_col = "futures",
  kernel_type = "ModifiedTukeyHanning",
  align_by = "seconds",
  align_period = 1
)

rk
```

---

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| realized_variance | <i>Estimate realized variance from efficient-price returns</i> |
|-------------------|----------------------------------------------------------------|

---

**Description**

Computes realized variance as the sum of squared intraday efficient-price returns over the overlapping trading period.

**Usage**

```
realized_variance(returns)
```

**Arguments**

|         |                                                                                                                                                                           |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| returns | A finite numeric vector of intraday efficient-price returns, normally obtained from changes in the common-price component estimated during an overlapping trading period. |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Details

During overlapping trading, the efficient price is obtained from the common-trend representation of the fitted VECM; see `vecm()` and `common_price()` for its construction. Following Dimpfl and Schweikert (2023), returns of this estimated common efficient price are used instead of selecting either the observed futures returns or the observed spot returns.

Once the efficient-price returns  $\Delta m_t$  have been estimated, the integrated variance can be calculated for any sufficiently long subperiod  $S_h$  of the overlapping trading period. If  $S_h$  contains  $n_h$  efficient-price returns, its variance is estimated as

$$\hat{\sigma}_{m,h}^2 = RV_h^{(n_h)} = \sum_{i=1}^{n_h} r_{h,i}^2,$$

where  $r_{h,i}$  is the  $i$ -th intraday return of the estimated common efficient price in subperiod  $S_h$ . Equivalently,  $r_{h,i}$  represents a change in the estimated common-price component.

This is the simple realized-variance estimator of Andersen et al. (2001) and Barndorff-Nielsen and Shephard (2002). In the absence of market-microstructure noise, it directly measures the integrated variance of that subperiod. In the presence of market-microstructure noise, ordinary realized variance may be biased.

The function itself only verifies that returns is numeric and finite and then calculates `sum(returns^2)`. It does not estimate the common price, construct returns, or assign observations to trading sessions. Those steps are performed by `common_price()` and `daily_cwis()` before this function is called.

For non-overlapping sessions, in which only one observed market price is available, the package uses `realized_kernel()` instead. The realized kernel is intended to reduce the influence of market-microstructure noise in the observed single-market returns.

## Value

A single non-negative numeric value equal to `sum(returns^2)`. It estimates the realized variation accumulated over the period represented by returns.

## Input requirements

returns should be a non-empty numeric vector containing returns from one price process, one trading day, and one clearly defined trading period. All elements must be finite; missing values, NaN, Inf, and -Inf are not permitted.

The sampling frequency should be consistent within the vector. If the inputs are log returns, the result is expressed in squared log-return units. If price changes are supplied instead, the result is expressed in squared price units and should not be interpreted as log-return variation.

When the input is derived from `common_price()`, returns can be constructed from changes in its cumulative permanent common-price component.

## References

- Andersen, T. G., Bollerslev, T., Diebold, F. X., and Labys, P. (2001). The distribution of realized exchange rate volatility. *Journal of the American Statistical Association*, 96(453), 42–55. doi:10.1198/016214501750332965
- Barndorff-Nielsen, O. E. and Shephard, N. (2002). Econometric analysis of realised volatility and its use in estimating stochastic volatility models. *Journal of the Royal Statistical Society: Series B*, 64(2), 253–280. doi:10.1111/14679868.00336

Dias, G. F. and Schweikert, K. (2022). Integrated variance estimation for assets traded in multiple venues. *SSRN Working Paper*, 1–48. <https://ssrn.com/abstract=4253762>

Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

### See Also

`common_price()` for extracting the permanent common-price component from a fitted VECM; `realized_kernel()` for noise-robust variance estimation in non-overlapping periods; `combine_weights()` for converting period-specific variance estimates into CWIS weights; `daily_cwis()` for the complete daily calculation.

### Examples

```
# A relative efficient-price path
efficient_price <- c(
  0,
  0.0010,
  0.0006,
  0.0014,
  0.0011
)

efficient_returns <- diff(efficient_price)

rv <- realized_variance(efficient_returns)
rv

# The function calculates the unadjusted sum of squared returns.
sum(efficient_returns^2)
```

---

|                    |                                                       |
|--------------------|-------------------------------------------------------|
| split_trading_days | <i>Split standardized observations by trading day</i> |
|--------------------|-------------------------------------------------------|

---

### Description

Splits standardized futures and spot-price observations into separate data frames for each trading day.

### Usage

```
split_trading_days(data)
```

### Arguments

|      |                                                                                                                                                                                                                     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data | A standardized data frame returned by <code>validate_and_standardize_input()</code> . It must contain the columns <code>datetime</code> , <code>trading_day</code> , <code>futures</code> , and <code>spot</code> . |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Details

Observations are grouped according to the `trading_day` column created by `validate_and_standardize_input()`. Each resulting data frame contains the observations belonging to one calendar day in the time zone used during standardization.

The function does not define intraday trading sessions, resample prices, or remove incomplete trading days. Intraday session assignment is performed separately by `split_trading_sessions()`.

Row names are reset within each daily data frame. The "tz" attribute of the input data is copied to every element of the returned list.

## Value

A named list of data frames, with one element for each trading day present in data. List names are the character representations of the corresponding trading dates. Each data frame retains the columns and chronological order of the standardized input and carries the same "tz" attribute.

## References

Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

## See Also

`validate_and_standardize_input()` for input validation and standardization; `split_trading_sessions()` for dividing one trading day into intraday sessions; `daily_cwis()` for estimating CWIS for a single trading day.

## Examples

```
example_data <- data.frame(
  timestamp = c(
    "2026-01-05 09:30:00",
    "2026-01-05 09:30:01",
    "2026-01-06 09:30:00",
    "2026-01-06 09:30:01"
  ),
  futures_price = c(5000, 5001, 5010, 5012),
  etf_price = c(500.0, 500.1, 501.0, 501.2)
)

standardized <- validate_and_standardize_input(
  data = example_data,
  datetime_col = "timestamp",
  futures_col = "futures_price",
  spot_col = "etf_price",
  spot_multiplier = 10,
  tz = "America/New_York"
)

daily_data <- split_trading_days(standardized)

names(daily_data)
daily_data[[1]]
```

---

split\_trading\_sessions

*Separate a trading day into market sessions*


---

## Description

Divides one standardized trading-day data frame into overlapping and single-market sessions according to local clock time.

## Usage

```
split_trading_sessions(
  day_data,
  boundaries = .default_session_boundaries(),
  tz = attr(day_data, "tz") %||% "America/New_York"
)
```

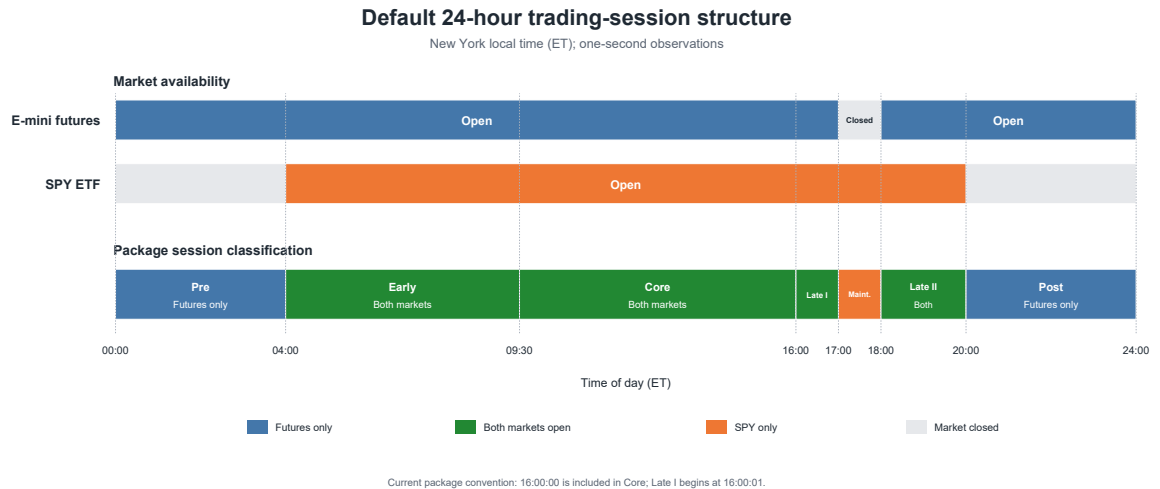
## Arguments

|            |                                                                                                                                                                                                                                                                                                            |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| day_data   | A standardized data frame for one trading day, normally one element returned by <a href="#">split_trading_days()</a> . It must contain datetime, trading_day, futures, and spot.                                                                                                                           |
| boundaries | A strictly increasing named numeric vector containing seconds from local midnight. It must contain, in order, the names overlap_open, core_open, core_close_exclusive, maintenance_start, overlap_resume, and overlap_close. All boundaries must be finite and lie strictly between 00:00:00 and 24:00:00. |
| tz         | A time-zone string used to convert timestamps to local clock time. By default, the "tz" attribute stored by <a href="#">validate_and_standardize_input()</a> is used. If that attribute is absent, "America/New_York" is used.                                                                             |

## Details

The function assigns every observation to one of seven non-overlapping component sessions. With the default boundaries, these sessions are:

### Trading-session structure:



- `futures_pre`: before 04:00:00;
- `overlap_pre`: from 04:00:00 to 09:30:00;
- `overlap_core`: from 09:30:00 to 16:00:01;
- `overlap_post_1`: from 16:00:01 to 17:00:00;
- `spot_maintenance`: from 17:00:00 to 18:00:00;
- `overlap_post_2`: from 18:00:00 to 20:00:00;
- `futures_post`: from 20:00:00 onward.

Except for the first and last sessions, intervals include their lower boundary and exclude their upper boundary. The combined overlap data frame is formed by chronologically joining `overlap_pre`, `overlap_core`, `overlap_post_1`, and `overlap_post_2`.

The default value of `core_close_exclusive` is 16:00:01. It was chosen for one-second observations so that 16:00:00 remains in `overlap_core` and the following second begins `overlap_post_1`. With sub-second data, observations between 16:00:00 and 16:00:01 are therefore also assigned to `overlap_core`. Users analysing millisecond data should verify that this convention is appropriate and, if necessary, supply sampling-frequency-appropriate boundaries.

Session membership is determined in `tz`. Specifying a different time zone changes the local clock time used for the allocation but does not change the underlying timestamp instants.

## Value

A named list containing the following data frames:

- `futures_pre`;
- `overlap_pre`;
- `overlap_core`;
- `overlap_post_1`;
- `spot_maintenance`;
- `overlap_post_2`;
- `futures_post`;
- `overlap`, containing all four overlapping-session components.

Each data frame retains the columns of `day_data`, has reset row names, and carries the time zone used by the function in its `"tz"` attribute. Sessions without observations are returned as empty data frames.

## References

Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:[10.1016/j.jbankfin.2023.106970](https://doi.org/10.1016/j.jbankfin.2023.106970)

## See Also

`validate_and_standardize_input()` for preparing the input data; `split_trading_days()` for creating daily data frames; `daily_cwis()` for calculating daily information shares from the resulting sessions.

## Examples

```
example_data <- data.frame(
  timestamp = c(
    "2026-01-05 03:59:59",
    "2026-01-05 04:00:00",
    "2026-01-05 09:30:00",
    "2026-01-05 16:00:00",
    "2026-01-05 16:00:01",
    "2026-01-05 17:00:00",
    "2026-01-05 18:00:00",
    "2026-01-05 20:00:00"
  ),
  futures_price = 5000:5007,
  etf_price = 500:507
)

standardized <- validate_and_standardize_input(
  data = example_data,
  datetime_col = "timestamp",
  futures_col = "futures_price",
  spot_col = "etf_price",
  spot_multiplier = 10,
  tz = "America/New_York"
)

day_data <- split_trading_days(standardized)[[1]]
sessions <- split_trading_sessions(day_data)

names(sessions)
vapply(sessions, nrow, integer(1))
```

---

validate\_and\_standardize\_input

*Validate and standardize CWIS input data*

---

## Description

Validates, scales, sorts, and standardizes two synchronized price series before they are used in the contribution-weighted information-share calculation.

The first series represents a futures market. The second represents the corresponding spot or cash-market price. In the empirical application of Dimpfl and Schweikert (2023), an ETF price is used as the cash-market proxy.

**Usage**

```
validate_and_standardize_input(
    data,
    datetime_col,
    futures_col,
    spot_col,
    spot_multiplier = 10,
    tz,
    drop_missing_datetime = TRUE
)
```

**Arguments**

|                                    |                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data</code>                  | A data frame containing one timestamp column and two synchronized numeric price columns representing the futures market and the corresponding spot, cash, or ETF market.                                                                                                                                                                                        |
| <code>datetime_col</code>          | A single character string naming the timestamp column. Its values must be character or inherit from POSIXt.                                                                                                                                                                                                                                                     |
| <code>futures_col</code>           | A single character string naming the numeric futures-price column. This series is assumed to be observed during futures-only trading sessions.                                                                                                                                                                                                                  |
| <code>spot_col</code>              | A single character string naming the numeric spot-, cash-, or ETF-price column. This series is assumed to be observed during the spot-only trading session.                                                                                                                                                                                                     |
| <code>spot_multiplier</code>       | A finite numeric scalar applied to the spot-price series during standardization, before logarithms are calculated. The multiplier should place the spot and futures prices on economically comparable scales. The default is 10, corresponding to the bundled S&P 500 futures and SPY example; use 1 when the supplied series are already on comparable scales. |
| <code>tz</code>                    | A single character string specifying the IANA/Olson time zone used to parse timestamps, assign local trading dates, and interpret intraday session boundaries. Defaults to "America/New_York", consistent with the U.S. market application considered in the paper.                                                                                             |
| <code>drop_missing_datetime</code> | A logical scalar. If TRUE, rows with missing or unparseable timestamps are removed. If FALSE, their presence causes an error.                                                                                                                                                                                                                                   |

**Details**

The function performs the following operations:

- verifies that `data` is a data frame containing the requested columns;
- requires both price columns to be numeric;
- parses character or POSIXt timestamps in `tz`;
- optionally removes rows with missing or unparseable timestamps;
- multiplies the spot-price series by `spot_multiplier`;
- rejects observed prices that are non-finite or non-positive;
- sorts observations chronologically;
- rejects duplicate timestamps; and
- assigns a local calendar trading date to every observation.

Individual missing prices are retained because one market may be inactive during a non-overlapping session. However, neither price series may be entirely missing. Missing prices required in a subsequent VECM or realized-kernel calculation can cause that trading day to fail.

`futures_col` and `spot_col` identify the economic roles of the two input series; the corresponding columns in data may have any names. For example, the NASDAQ-100 application may use an E-mini futures column for `futures_col` and a QQQ ETF column for `spot_col`.

`spot_multiplier` is product-specific. The default value of 10 corresponds to the bundled S&P 500 example. Users should set it to 1 when the two input series are already on the intended scales, or supply another appropriate conversion factor.

`tz` determines timestamp interpretation, calendar-day assignment, and subsequent trading-session allocation. A valid Olson time-zone name such as "America/New\_York" is recommended. Fractional seconds are supported when they are present in character timestamps.

## Value

A data frame with observations sorted by `datetime` and row names reset. It contains:

**datetime** A POSIXct timestamp represented in `tz`.

**trading\_day** A Date derived from the local calendar date in `tz`.

**futures** The standardized futures-price series.

**spot** The spot-price series after multiplication by `spot_multiplier`.

The selected time-zone string is also stored in the "tz" attribute of the returned data frame.

## Input requirements

data must be a data frame containing one timestamp column and two numeric price columns. Their names are supplied through `datetime_col`, `futures_col`, and `spot_col`.

The timestamp column must be either a character vector or an object inheriting from POSIXt. Character timestamps are interpreted in `tz`. Supported representations include:

- "YYYY-MM-DD HH:MM:SS";
- "YYYY/MM/DD HH:MM:SS";
- "YYYY-MM-DDTHH:MM:SS";
- "YYYY-MM-DDTHH:MM:SSZ".

Fractional seconds are supported, for example "2026-01-05 09:30:00.125".

Missing or unparseable timestamps are removed when `drop_missing_datetime = TRUE`. If `drop_missing_datetime = FALSE`, their presence causes an error. The remaining observations are sorted chronologically. Duplicate timestamps are not combined and cause the function to stop.

Both price columns must be numeric. Individual prices may be missing, but neither price series may be entirely missing. Observed prices must be finite and strictly positive after applying `spot_multiplier`. Missing prices are retained and are not imputed.

## Dates and time zone

`tz` determines how character timestamps are interpreted and how `trading_day` is assigned. It should normally be a valid IANA/Olson time-zone name, such as "America/New\_York", "Europe/Berlin", "Europe/London", "Asia/Shanghai", or "UTC". Available names can be inspected with `OlsonNames()`.

The `trading_day` column represents the calendar date of each observation in `tz`. Dates absent from the input are not inserted. Similarly, a date from which all observations are removed during timestamp validation will not appear in the returned data.

Time-zone abbreviations such as "EST" or "CET" are best avoided because they may not apply daylight-saving-time rules as intended.

## References

Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

## See Also

`split_trading_days()` for dividing the standardized data by local date, `split_trading_sessions()` for assigning intraday sessions, and `cwis()` for the complete multi-day CWIS calculation.

## Examples

```
example_data <- data.frame(
  timestamp = c(
    "2026-01-05 09:30:01.250",
    "2026-01-05 09:30:00.500",
    NA_character_
  ),
  futures_price = c(5001, 5000, 5002),
  etf_price = c(500.1, 500.0, 500.2)
)

standardized <- validate_and_standardize_input(
  data = example_data,
  datetime_col = "timestamp",
  futures_col = "futures_price",
  spot_col = "etf_price",
  spot_multiplier = 10,
  tz = "America/New_York"
)

standardized
attr(standardized, "tz")
```

---

vecm

*Estimate a VECM for the overlapping trading period*

---

## Description

Estimates a bivariate Johansen vector error-correction model (VECM) for synchronized log futures and spot prices observed while both markets are trading.

## Usage

```
vecm(log_prices, K = 10, type = "trace", ecdet = "none", spec = "transitory")
```

## Arguments

|            |                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| log_prices | A numeric two-column matrix or data frame containing log futures prices in the first column and log spot prices in the second.                                                                                                                                                                                                                                                                      |
| K          | Integer lag order of the variables in the underlying level VAR passed to <code>urca::ca.jo()</code> . The corresponding VECM contains $K - 1$ lags of first differences. $K$ must be at least 2 for use with the downstream CWIS functions. The default is 10. Users may choose another value according to the sampling frequency, number of available observations, and lag-selection diagnostics. |
| type       | Character string specifying the Johansen cointegration-rank test: "trace" for the trace test or "eigen" for the maximum-eigenvalue test.                                                                                                                                                                                                                                                            |
| ecdet      | A character string specifying the deterministic term in the cointegration relation: "none", "const", or "trend". Defaults to "none", meaning that no intercept is included in the cointegration relation.                                                                                                                                                                                           |
| spec       | Character string specifying the VECM parameterization: "transitory" or "longrun".                                                                                                                                                                                                                                                                                                                   |

## Details

When two markets simultaneously trade claims on the same underlying asset, the law of one price implies that their prices share a common long-run value. Observed prices may nevertheless deviate temporarily because of market-specific microstructure effects and differences in adjustment speed (Hasbrouck, 1995; Dimpfl and Schweikert, 2023).

These common long-run and market-specific short-run dynamics can be represented by a cointegrated vector error-correction model (VECM). For the default transitory specification, the model estimated by `urca::ca.jo()` can be summarized as

$$\Delta p_t = \Pi p_{t-1} + \sum_{i=1}^{K-1} \Gamma_i \Delta p_{t-i} + d_t + u_t,$$

where  $p_t$  contains the futures and spot log prices,  $\Pi = \alpha\beta'$  is the long-run coefficient matrix,  $\beta$  is the cointegrating vector,  $\alpha$  contains the adjustment coefficients,  $\Gamma_i$  contains the short-run coefficient matrices,  $d_t$  represents the deterministic terms, and  $u_t$  is a vector of reduced-form white-noise innovations satisfying  $E(u_t) = 0$  and

$$\Omega = E(u_t u_t') = \text{Var}(u_t).$$

The fitted model supplies the cointegrating vectors, adjustment coefficients, short-run coefficients, innovations, and innovation covariance information used in the subsequent price-discovery analysis. In particular, `common_price()` uses the VECM to recover innovations in the permanent common price, while `his()` uses its long-run impact and innovation covariance matrices to calculate Hasbrouck information shares.

With `spec = "transitory"`, the error-correction term is based on  $p_{t-1}$ , and the short-run coefficients describe transitory effects. With `spec = "longrun"`, `urca::ca.jo()` uses its alternative long-run parameterization. The cointegration matrix and the corresponding inference are unchanged, but the short-run coefficient matrices differ.

The function conducts the Johansen procedure but does not automatically select a lag order or impose a cointegration rank. The intended downstream CWIS calculation assumes that the two prices are linked by one economically meaningful common stochastic trend. Users should assess whether this assumption is appropriate for their data.

## Value

An S4 object of class `ca.jo` returned by `urca::ca.jo()`. The object contains the Johansen test statistics, eigenvalues, cointegrating vectors, loading coefficients, short-run coefficient matrices, residual-related matrices, and estimation settings. The two input variables are stored under the standardized names `futures` and `spot`.

The returned object can be passed to `his()` to calculate Hasbrouck information shares and to `common_price()` to estimate innovations in the permanent common price.

## Lag order and relation to the referenced methodology

Dimpfl and Schweikert (2023) use a high-resolution VECM with a heterogeneous-autoregressive structure for the short-run coefficients. In their specification,  $\delta$  denotes the number of time scales and  $k_i$  defines the range of each scale. At a sampling frequency of 10 milliseconds, they set  $\delta = 4$ ,  $k_1 = 1$ ,  $k_2 = 10$ ,  $k_3 = 100$ , and  $k_4 = 1000$ .

The four coefficient matrices  $\Gamma^{(1)}, \dots, \Gamma^{(4)}$  are applied to price changes aggregated over lag ranges 1, 2–10, 11–100, and 101–1000 observations, respectively. The final range therefore extends to a maximum lag horizon of ten seconds while only four time-scale coefficient matrices are estimated.

The present function instead estimates the standard Johansen VECM provided by `urca::ca.jo()`. Here,  $K$  is the lag order of the underlying level VAR, and the corresponding VECM contains  $K - 1$  separately estimated first-difference lag matrices.

The time span represented by  $K$  depends on the sampling interval. With one-second observations,  $K = 10$  represents a maximum level-VAR lag of approximately ten seconds, whereas with 10-millisecond observations it represents only approximately 100 milliseconds.

Consequently,  $K$  should be selected for the sampling frequency and data used in the present implementation rather than copied mechanically from the reference paper. Larger values allow richer short-run dynamics but require more observations and increase estimation uncertainty and computational cost.  $K$  must be at least 2 because the downstream CWIS calculation requires at least one lag of first differences.

The function therefore implements the standard-VECM component of the CWIS framework but is not an exact replication of the paper's high-resolution HAR-VECM.

## Input requirements

`log_prices` must contain exactly two columns, ordered as futures followed by spot. Values must already be expressed as logarithmic prices; the function does not transform price levels to logarithms.

Rows should represent synchronized, chronologically ordered observations from periods in which both markets are trading. All values must be numeric and finite, and both price series must vary over the estimation sample. Missing values are not permitted.

The number of observations must be greater than  $K + 2$ . This is only a minimum computational requirement. Substantially more observations may be needed for reliable estimation, especially when a large  $K$  is used.

## References

- Johansen, S. (1988). Statistical analysis of cointegration vectors. *Journal of Economic Dynamics and Control*, 12(2-3), 231-254. doi:10.1016/01651889(88)900413
- Johansen, S. (1991). Estimation and hypothesis testing of cointegration vectors in Gaussian vector autoregressive models. *Econometrica*, 59(6), 1551-1580. doi:10.2307/2938278

Hasbrouck, J. (1995). One security, many markets: Determining the contributions to price discovery. *The Journal of Finance*, 50(4), 1175–1199. doi:10.2307/2329348

Hasbrouck, J. (2021). Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 395–430. doi:10.1093/jjfinec/nbz027

Buccheri, G., Bormetti, G., Corsi, F., and Lillo, F. (2021). Comment on: Price discovery in high resolution. *Journal of Financial Econometrics*, 19(3), 439–451. doi:10.1093/jjfinec/nbz008

Dimpfl, T. and Schweikert, K. (2023). Information shares for markets with partially overlapping trading hours. *Journal of Banking & Finance*, 154, 106970. doi:10.1016/j.jbankfin.2023.106970

## See Also

`urca::ca.jo()` for the underlying Johansen procedure; `his()` for Hasbrouck information shares; `common_price()` for innovations in the common efficient price; `daily_cwis()` for the complete daily calculation.

## Examples

```
set.seed(123)

n <- 300
common_trend <- cumsum(rnorm(n, sd = 0.002))
futures_noise <- rnorm(n, sd = 0.0002)
spot_noise <- rnorm(n, sd = 0.0003)

simulated_log_prices <- cbind(
  futures = log(5000) + common_trend + futures_noise,
  spot = log(5000) + common_trend + spot_noise
)

# K = 2 is used here to keep the simulated example compact. Users may
# choose a larger value when supported by the data and diagnostics.
fit <- vecm(
  log_prices = simulated_log_prices,
  K = 2,
  type = "trace",
  ecdet = "none",
  spec = "transitory"
)

# Johansen trace statistics and critical values
fit@teststat
fit@cval
```

# Index

\* **datasets**  
    example\_sp500\_futures\_spot\_oneweek\_24h\_1sec, 25

aggregate\_cwis, 5  
aggregate\_cwis(), 3, 4, 11, 17, 20, 24

base::format.Date(), 30

combine\_weights, 8  
combine\_weights(), 3, 7, 17, 20, 23, 24, 28, 34, 36

common\_price, 11  
common\_price(), 3, 11, 17, 20, 22, 24, 28, 35, 36, 44–46

cwis, 15  
cwis(), 3–5, 7, 22, 30, 43

daily\_cwis, 21  
daily\_cwis(), 3–5, 7, 11, 14, 17, 19, 20, 27, 28, 34–37, 40, 46

example\_sp500\_futures\_spot\_oneweek\_24h\_1sec, 25

graphics::legend(), 31  
graphics::par(), 31  
graphics::plot(), 30, 31

highfrequency::listAvailableKernels(), 16, 22, 32, 34  
highfrequency::rKernelCov(), 16, 19, 22, 32, 34

his, 26  
his(), 3, 7, 11, 14, 17, 20, 22, 24, 44–46

plot.cwis\_result, 29  
plot.cwis\_result(), 20

priceDiscoverY  
    (priceDiscoverY-package), 2  
priceDiscoverY-package, 2

realized\_kernel, 31  
realized\_kernel(), 3, 7, 9, 11, 17, 20, 22, 24, 35, 36

realized\_variance, 34  
realized\_variance(), 7, 9, 11, 14, 17, 20, 22, 24, 34

split\_trading\_days, 36  
split\_trading\_days(), 17, 22, 38, 40, 43  
split\_trading\_sessions, 38  
split\_trading\_sessions(), 16–18, 20, 22, 24, 33, 34, 37, 43

urca::ca.jo(), 4, 16, 19, 22, 24, 26, 44–46  
urca::cajorls(), 11, 14

validate\_and\_standardize\_input, 40  
validate\_and\_standardize\_input(), 17, 20, 22, 24, 36–38, 40

vecm, 43  
vecm(), 3, 11, 13, 14, 17, 20, 22, 24, 26, 28, 35

xts::xts(), 32